Computers & Industrial Engineering 182 (2023) 109436

Contents lists available at ScienceDirect

Computers & Industrial Engineering

journal homepage: www.elsevier.com/locate/caie

An aircraft final assembly line balancing problem considering resource constraints and parallel task scheduling

Zhongkai Bao, Lu Chen *, Kejun Qiu

School of Mechanical Engineering, Shanghai Jiao Tong University, No. 800, Dongchuan Road, Minhang District, Shanghai, 201100, China

ARTICLE INFO

Keywords:
Aircraft final assembly line
Assembly line balancing problem
Resource constraints
Construction and improvement heuristic approach
Priority rule

ABSTRACT

In this study, the balancing problem of an aircraft final assembly line (AFAL) is investigated. The decision is to assign all the assembly tasks to a given number of workstations, with the objectives to balance the workstation cycle time and to minimize the total resource investment cost. An integer linear programming model is developed to formulate the problem. A construction and improvement based two-stage heuristic approach is proposed to solve this problem. In the first stage, an initial task assignment is constructed based on rules that are evolved by a multi-dimensional archive of phenotypic elites algorithm. In the second stage, an improvement heuristic, consisting of several customized neighborhood operators, is developed to improve the initial task assignment. The proposed approach is compared with the existing methods and its variants. Computational experiments and a real case study reveal the superiority of our proposed approach.

1. Introduction

China's commercial aircraft industry has experienced rapid development in the past ten years. The aircraft manufacturing is transiting from trial production to batch production. As the final stage of producing aircraft, the aircraft final assembly line (AFAL) is responsible for performing assembly tasks such as installation and testing of aircraft systems at workstations, which consumes up to 50% of total production time and more than 20% of total production cost (Caggiano et al., 2016). Drawing on the successful experience of Boeing company, Commercial Aircraft Corporation of China has applied the moving assembly line to improve aircraft production efficiency and to meet the growing demand. However, balancing of the AFAL is a big challenge. Unlike the assembly lines with high automation (e.g., automobile assembly line), manual assembly tasks account for a majority (nearly 80%) of all tasks in aircraft assembly. Multiple resources (workers and tools) are required to perform a task, and the workers can operate simultaneously on the same workpiece of aircraft.

This paper focuses on the aircraft final assembly line balancing problem considering resource constraints (AALBP-RC) derived from a practical AFAL. The AFAL is a moving assembly line consisting of a certain number of workstations. A large number of assembly tasks need to be assigned to workstations under predefined precedence relations. Aircrafts pass through each workstation after a fixed cycle time to complete the assembly tasks. A limited resource is distributed along the assembly line to execute the tasks, which imposes a strong constraint on the balancing of the AFAL. Moreover, a large number of tasks can be executed in parallel, which makes the calculation of actual cycle time very challenging.

In the investigated AALBP-RC, the objectives are to balance the actual cycle times of workstations and to minimize the total resource investment cost. Special attention needs to be paid to incorporate task attributes (including resource factors) of aircraft assembly into task assignment and thus to reduce computational burden of the solution technique. Moreover, task scheduling is adopted as the auxiliary decision to evaluate the cycle time and resource allocation of each workstation.

The remainder of the paper is structured as follows. A literature review is presented in Section 2. The mathematical model of the problem is described in Section 3. The two-stage heuristic approach is developed in Section 4. Section 5 reports the computational results. Finally, Section 6 concludes the paper and proposes future research directions.

2. Literature review

The operation management of aircraft assembly lines mainly focuses on the line balancing problem and the task scheduling problem. In this section we review the literature on the assembly line balancing problem, and on the scheduling of aircraft assembly lines respectively. Finally, the contributions of this paper are discussed.

* Corresponding author.
E-mail addresses: bzk226@sjtu.edu.cn (Z. Bao), chenlu@sjtu.edu.cn (L. Chen), sjtu.10399-lord@sjtu.edu.cn (K. Qiu).

https://doi.org/10.1016/j.cie.2023.109436
Received 8 March 2023; Received in revised form 26 May 2023; Accepted 7 July 2023
Available online 10 July 2023
0360-8352/© 2023 Elsevier Ltd. All rights reserved.

2.1. Assembly line balancing problem

The assembly line balancing problem (ALBP) was firstly formulated by Salveson in 1955 (Eghtesadifard et al., 2020). It is the process of assigning assembly tasks to workstations with respect to some objective and in accordance with occurrence constraints and precedence constraints. The ALBP can be divided into two categories: (1) simple assembly line balancing problem (SALBP) where a single product is processed in a serial assembly line with certain input parameters; (2) generalized assembly line balancing problem (GALBP), which is the extended SALBP by integrating more constraints like U-shaped lines, mixed model and uncertain factors. The SALBP can be further divided into three categories: (1) type-1 of SALBP aims to minimize the number of workstations under a given cycle time; (2) type-2 of SALBP tries to minimize the cycle time under a given number of workstations; (3) type-E of SALBP pursues the maximization of line efficiency with cycle time and the number of workstations variable. Boysen et al. (2007) also proposed a classification scheme of the ALBP based on three elements: the characteristics of the precedence graph, the characteristics of the station and line and the objectives. Comprehensive reviews related to the classification and development of ALBPs can be found in the surveys by Eghtesadifard et al. (2020) and by Battaïa and Dolgui (2013). Recently, some realistic factors have been incorporated into the ALBPs to obtain more effective and efficient balancing solutions, such as ergonomics (Ozdemir et al., 2021), energy consumption (Liu et al., 2021), learning and fatigue effects (Eslamipoor & Nobari, 2021) and resource constraints (Alakaş, 2021). Since this paper investigates the line balancing problem of aircraft final assembly considering resource constraints, we concentrate on reviewing the ALBP in aircraft assembly and the ALBP considering resource constraints.

The researches on the balancing of aircraft assembly line are rather rare. Motivated by the moving assembly line of Hawker Beechcraft, Abdinnour (2011) developed a linear programming based greedy heuristic to determine the number of stations and the assignment of tasks to workstations and zones. Ríos et al. (2012) studied the balancing of an AFAL from AIRBUS Military in the industrialization stage. The workstations were constructed according to the technological criteria. Further on, Borreguero et al. (2015) extended the solution approach of scheduling to the aircraft production stage. Xin et al. (2015) developed an adaptive binary particle swarm optimization algorithm to balance the workload among workstations and tasks, and to minimize the human cost in aircraft assembly lines. However, these works either ignore task assignment decisions, or overlook resource constraints.

Resources like workers, tools, and machines that provide production capabilities are necessary for executing assembly tasks. The resources are limited and need to be efficiently allocated to workstations. However, many existing researches neglect the influence of resources on ALBPs. Here we review the ALBP considering resource constraints that is essentially a branch of GALBP. Ağpak and Gökçen (2005) introduced a resource constrained assembly line balancing problem (RCALBP) whose objective was to assign tasks and resources to workstations such that the number of resources was minimized. In their study, tasks using different resources are not allowed at the same workstation. Each task can only be executed by one resource type or by either of two resource types. Ogan and Azizoglu (2015) proposed a branch-and-bound method to address such a problem in a U-shaped assembly line, where the total equipment cost was minimized. Quyen et al. (2017) developed a two-stage heuristic approach based on genetic algorithm and priority rules for a RCALBP in footwear manufacturing plant. Chen et al. (2018) studied multi-manned RCALBP and developed a hybrid genetic algorithm to minimize the number of workstations, operators and resources simultaneously. Corominas et al. (2011) generalized the work of Ağpak and Gökçen (2005) by allowing more flexible types of resource requirement: (1) one task may require multiple types/units of resource; (2) one task may be processed by combinations of alternative and/or concurrent resources. Alakaş (2021) established a constraint programming model to solve the generalized RCALBP. Jin et al. (2022) studied a robust generalized RCALBP with uncertain task times. A constraint generation technique-based method and an enhanced whale optimization algorithm were developed to minimize the maximum regret in all the scenarios.

In addition to the RCALBP, the ALBPs considering various practical resource factors have been studied in the literature. Şahin and Kellegöz (2019) introduced a resource investment and balancing problem of a parallel multi-manned assembly line. The objective was to minimize the total cost incurred by resources and opened workstations. A particle swarm optimization algorithm based on a modified serial schedule generating scheme was designed to solve the problem. Pinhão et al. (2022) studied the integrated line balancing and scheduling problem of aircraft engines assembly lines considering the assignment of multi-skilled operators to tasks. Four integer programming models with different objective functions were developed to solve the problem hierarchically. Pitakaso et al. (2021) designed a variable neighborhood strategy adaptive search algorithm to solve the type-2 of SALBP with the constraint imposed by the number of machine's types. Zhang et al. (2021) integrated the balancing and preventive maintenance problems in resource-dependent assembly lines. A memetic algorithm with multi-attribute solution representation was designed to tackle the problem. Fang et al. (2020) studied a multi-robotic disassembly line balancing problem considering limited resources including energy resources. A multi-objective optimization approach with the epsilon method was proposed to address the problem.

2.2. Scheduling of aircraft assembly lines

The aircraft assembly scheduling problem has received great attention since the demand for scheduling is more frequent in actual production and it also involves an important decision concerning task starting/ending time. Project scheduling theories are often adopted for the scheduling of aircraft assembly. Ren et al. (2020) abstracted the scheduling of aircraft assembly as a resource constrained project scheduling problem (RCPSP) with the resource transfer time. A branch-and-bound embedded genetic algorithm was developed to solve the problem. Arkhipov et al. (2018) studied a RCPSP with economic and ergonomic constraints in an aircraft assembly line. An exact approach based on constraint programming and integer linear programming was developed to tackle this problem. Motivated by an aircraft moving assembly line, Lu et al. (2019) introduced a resource investment project scheduling problem considering project splitting and time windows of resource availability, and then proposed a two-stage heuristic combining genetic algorithm and branch-and-bound algorithm.

The scheduling of aircraft assembly also involves the resource allocation decision. Ren and Lu (2019) developed an integrated nested optimization algorithm to determine the task scheduling, multi-skilled worker allocation and project splitting simultaneously in an aircraft moving assembly line. Fang et al. (2021) developed a modified non-dominated sorting genetic algorithm to address the worker allocation problem in an aircraft assembly line with multi-stage workstation.

Some other researches consider uncertainties in the scheduling of aircraft assembly. Urgo (2019) introduced the uncertainty of resource availability into the scheduling of an aircraft assembly line, and then developed a branch-and-bound algorithm. Zhu et al. (2021) introduced a rescheduling problem caused by uncertain material delivery in an aircraft assembly line. A machine learning-based reactive scheduling method was developed to determine the rescheduling policy. Lovato et al. (2022) proposed a constraint programming-based model with staircase criteria inject to address the rescheduling problem in aircraft assembly lines.

2.3. Contributions of our paper

The existing researches on the assembly line balancing either ignore the impact of resource constraints on line balance, or fails to consider the influence of numerous parallel tasks on the calculation of actual cycle time. Thus, there exists a gap between current research and real practice. This paper aims to fill in this gap to effectively address the problem. Our main contributions are as follows:

(1) A bi-objective integer linear programming model is firstly developed to formulate the problem. The objectives are then transformed into a single objective function with both objective functions optimized. As a result, the problem can be solved directly by a commercial Solver.
(2) With the existence of a large number of parallel tasks, evaluation of the cycle time is very challenging. Therefore, the first stage of our approach proposes a hyper-heuristic to evolve an archive of diverse and high-performing priority rules that can be applied to construct an efficient task assignment. Then, several customized neighborhood operators are designed to effectively adjust the task assignment in the second stage of our approach. Based on the priority rules injected with problem attributes, the extensive iterative search in decision space can be avoided and thus the efficiency of the solution technique can be improved.
(3) A large number of instances are generated based on the real aircraft assembly line. The proposed approach is compared with its variants and an existing approach. A real case study is conducted to assess the effectiveness of the proposed approach in real production.

3. Problem formulation

In this section, we present the definition and formulation for the investigated AALBP-RC, followed by the transformation of the objective functions.

3.1. Problem definition

In the addressed AALBP-RC, the set of assembly tasks is denoted as $J = 0, 1, \ldots, H$, where 0 and H are dummy tasks with zero processing times and non-resource requirements. The processing time of each non-dummy task j ($j \in J$) is represented by p_j. I_j is defined as the set of immediate predecessors of task j ($j \in J$), indicating that j could be started only after all the tasks in I_j are completed. All the non-dummy tasks are required to be assigned to a set of workstations K. The tasks in each workstation must respect the precedence relations and be finished within a given time period defined as cycle time (C). For a given workstation, the actual cycle time is the maximum of the completion times of the assigned tasks, which needs to be determined by task scheduling.

A set of renewable resources R is required through the assembly, each of which ($r \in R$) corresponds to a unit resource cost c_r. There are two types of resources, namely worker resource and tool resource. Tool resource refers to the specialized tools or equipment for fixing, positioning, and testing etc. The execution of each non-dummy task has a requirement of tool resource and human resource simultaneously. Define w_{jr} as the requirement for resource r ($r \in R$) by task j ($j \in J$).

The decisions to be made are: (1) the assignment of tasks to workstations; (2) the capacity of each type of resource invested to the assembly line. The objectives are to balance the actual cycle times of workstations and to minimize the total resource investment cost. The auxiliary decision is the schedule of tasks to obtain task starting times and thus to evaluate the cycle time and resource allocation of each workstation.

To illustrate the problem, an instance with five non-dummy tasks and one type of resource is created. The cycle time and the number of workstations are set to be six and two, respectively. The unit cost of resource is assumed as 1. The resource requirement and processing time of each task are given in Table 1.

Table 1
Basic information of the instance.

j	Resource requirement (w_{jr})	Processing time (p_j)
0	0	0
1	1	3
2	3	2
3	2	2
4	1	4
5	2	1
6	0	0

Fig. 1 depicts the process of obtaining a feasible solution of the instance. The numbers in the circles denote the serial number of tasks. From Fig. 1, tasks 1, 2 and 5 are assigned to the workstation 1, and tasks 3 and 4 are assigned to the workstation 2. Note that precedence relations are satisfied in each workstation. A feasible schedule of the tasks along with the required resource investment are presented. The maximum resource requirement of the whole assembly line is four and occurs in the time interval $[3, 5]$. Both the actual cycle times of the two workstations are six, meaning that the assembly line is balanced.

Compared to the simple instance, the actual production problem is much more challenging since it involves a greater number of tasks, more types of resources, a longer cycle time and thus a significantly larger decision space.

3.2. Mathematical formulation

Denote $D = \{0, 1, \ldots, C\}$ to be the set of discrete time and M to be a big number. Decision variables are defined as:

x_{jk}	=1 if task j is assigned to workstation k and 0 otherwise, $j \in J$, $k \in K$
q_r	Capacity of resource r that represents the maximal requirement of resource r In all time units, $r \in R$
T_k	Actual cycle time of workstation k, $k \in K$
F	Total resource investment cost
SI	Smoothness index
s_j	Auxiliary variable, starting time of task j, $j \in J$
y_{jd}	Auxiliary variable, equaling to 1 if task j is executed at the time d and 0 Otherwise, $j \in J$, $d \in D$

The mathematical model of the AALBP-RC is formulated as below.

$$\min SI = \sqrt{\frac{1}{H}\sum_{k\in K}(C - T_k)^2} \tag{1}$$

$$\min F = \sum_{r\in R} c_r q_r \tag{2}$$

subject to

$$\sum_{k\in K} x_{jk} = 1, \forall j \in J\backslash\{0, H\} \tag{3}$$

$$\sum_{k=t}^{|K|} x_{ik} \le \sum_{k=t}^{|K|} x_{jk}, \forall j \in J\backslash\{0, H\}, \forall i \in I_j, \forall t \in K \tag{4}$$

$$s_j - s_i \ge p_i - M(2 - x_{ik} - x_{jk}), \forall j \in J\backslash\{0, H\}, \forall i \in I_j, \forall k \in K \tag{5}$$

$$C \ge s_j + p_j, \forall j \in J \tag{6}$$

$$q_r \ge \sum_{j\in J} w_{jr} y_{jd}, \forall d \in D, \forall r \in R \tag{7}$$

$$\sum_{d\in D} y_{jd} = p_j, \forall j \in J \tag{8}$$

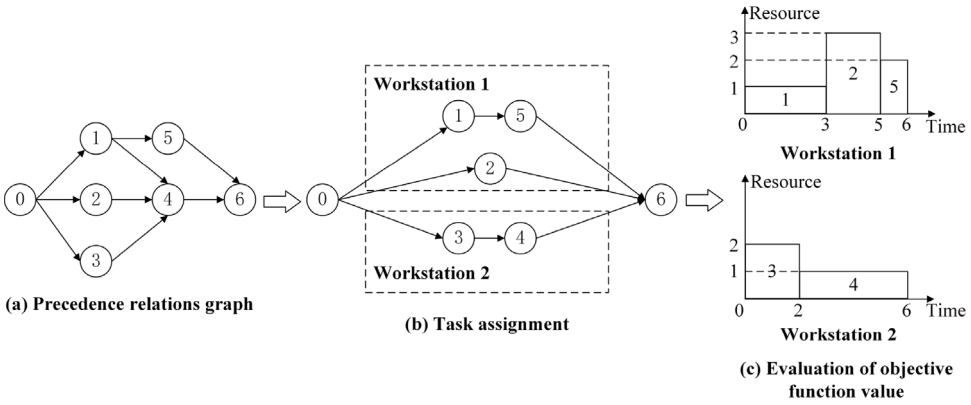


Fig. 1. A feasible solution of the instance.

$$d - s_j - p_j + 1 \le M(1 - y_{jd}), \forall j \in J, \forall d \in D \quad (9)$$

$$s_j - d \le M(1 - y_{jd}), \forall j \in J, \forall d \in D \quad (10)$$

$$y_{jd} \in \{0,1\}, \forall j \in J, \forall d \in D \quad (11)$$

$$x_{jk} \in \{0,1\}, \forall j \in J, \forall k \in K \quad (12)$$

$$T_k \in \mathbb{N}, \forall k \in K \quad (13)$$

$$s_j \in \mathbb{N}, \forall j \in J \quad (14)$$

$$q_r \in \mathbb{N}, \forall r \in R \quad (15)$$

The first objective function (1) minimizes smoothness index (SI). The SI measures the difference of the actual cycle times of workstations. A lower SI value implies a better balancing line. The second objective function (2) minimizes the total resource investment cost F. Constraints (3) ensure that each task can only be assigned to one workstation. Constraints (4) represent the precedence constraints for task assignment, which means that the immediate predecessor tasks of task j cannot be assigned to the later workstations. Constraints (5) guarantee that the precedence relations are respected in each workstation. Constraints (6) impose the cycle time restriction for all the assembly tasks. Constraints (7) denote that the capacity of each type of resource must satisfy the resource requirements at any time unit. Constraints (8) define the relationship between processing time and the auxiliary variable that represents the schedule of executing tasks. Constraints (9) and (10) prevent the tasks from being interrupted while in execution. Constraints (11)–(15) define the properties of the decision variables.

3.3. Transformation of the objective function

This section simplifies the model by transforming the objective function.

Proposition 1. *Minimizing the total resource investment cost ensures that the smoothness index is optimized.*

Proof. After the tasks have been assigned to workstations, the capacity of each type of resource and actual cycle time of each workstation are calculated by scheduling the tasks in a given cycle time. Since the resources considered here are renewable, the resource capacity is equal to the maximum resource requirement in all time units. To minimize the resource capacity, the parallel tasks need to be converted into serial executions as far as possible. An extreme case is that when the given cycle time is long enough, all the parallel tasks will be executed serially to achieve the minimum resource investment. Hence, when taking the minimization of total resource investment cost as the objective function, the length of the given cycle time will be fully utilized, which can make the actual cycle time of each workstation as close as possible to the given cycle time and thus to optimize the smoothness index.

Therefore, minimizing the total resource investment cost can promote the balance of cycle times between the workstations. To avoid computational complexity brought by multi-objective optimization, we therefore consider a single objective function that is to minimize total resource investment cost. Thus, the original formulation (1–15) is simplified as below.

$$\min F = \sum_{r \in R} c_r q_r$$

subject to (3)–(15)

4. Solution approach

In this section, a construction and improvement based two-stage heuristic (CITH) approach is developed to solve the AALBP-RC. As depicted in Fig. 2, the CITH includes two stages: task assignment construction and task assignment improvement. In the first stage, the multi-dimensional archive of phenotypic elites (ME) based hyper-heuristic is developed to generate a priority rule. With the priority rule, an initial task assignment can be generated. In the second stage, a variable neighborhood search-based improvement heuristic (IH) is developed to improve the initial task assignment.

The multi-dimensional archive of phenotypic elites (ME) algorithm introduced by Mouret and Clune (2015) possesses a strong ability of global search. It produces an archive of diverse and high-performing solutions (phenotypic elites) based on a discretized multi-dimensional feature map. Only two studies related to the ME based hyper-heuristic were found in the literature. Sánchez et al. (2021) applied the ME to search for the sequence-based selection hyper-heuristic for solving the balanced partition problem. Chand et al. (2022) developed a ME based hyper-heuristic to evolve the rules for the RCPSP. In this paper, the phenotypic elites refer to excellent priority rules.

4.1. Task assignment construction

In this section, the first stage of the proposed approach CITH is described. The representation of priority rule is firstly introduced. Then the ME based hyper-heuristic is used to generate priority rules for the construction of task assignment.

4.1.1. Rule representation

A tree structure is used to represent a priority rule. Actually, such a tree indicates a formula that combines task attributes (also referred to as terminal symbols) and arithmetic operator. In this context, the priority value of a task refers to the priority selected to be assigned to a workstation.

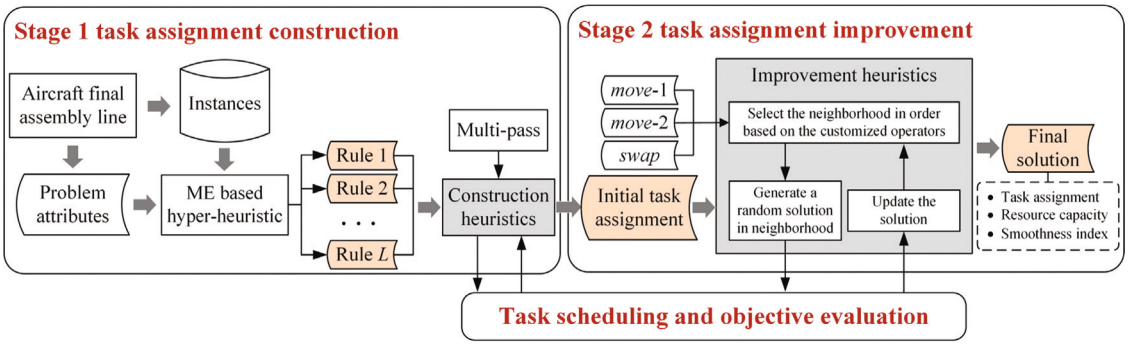


Fig. 2. Framework of the proposed CITH approach.

Table 2
Task attribute set.

Number	Attribute	Description
1	RF	Resource factor that represents the density of different types of resource required by a task
2	AR	Average resource requirement of a task
3	MAR	Maximum requirement of all the types of resource for a task
4	MIR	Minimum requirement of all the types of resource for a task
5	WL	Workload of a task that equals to the product of resource requirement and processing time
6	LS	Latest starting time of a task when the completion time is set as the length of critical path in the precedence graph
7	LF	Latest finishing time of a task when the completion time is set as the length of critical path in the precedence graph
8	NP	Number of predecessors of a task
9	NS	Number of successors of a task
10	PW	Total task processing times of successors of a task
11	EW	Earliest workstation that can be assigned to for a task
12	LW	Latest workstation that can be assigned to for a task
13	SW	Slack workstation that is obtained by subtracting EW from LW for a task

The ME evolves the tree (rule) by operating on the predefined sets of attributes and arithmetic operators. Seven basic operators are defined: (1) addition (+), (2) subtraction (−), (3) multiplication (×), (4) division (\), (5) negative (neg), (6) maximization (max) and (7) minimization (min). As shown in Table 2, thirteen attributes that capture the characteristics of aircraft assembly tasks are considered. Specifically, attributes 1–4 reflect the resource factor of a task; attribute 5 represents the workload of a task; attributes 6 and 7 are calculated based on the critical path method; attributes 6–13 are related to the precedence relations among tasks. Attributes 11–13 also contains information with respect to workstation (refer to Appendix A for more information). Normalization is conducted to make the priority values range between zero and one.

Fig. 3 provides the tree structure of a sample rule. The leaf nodes represent the task attributes (AR, NS and LW), and the non-leaf nodes indicate the arithmetic operators (addition and subtraction). Thus, the rule is "AR-NS + LW" that calculates the corresponding value of each task.

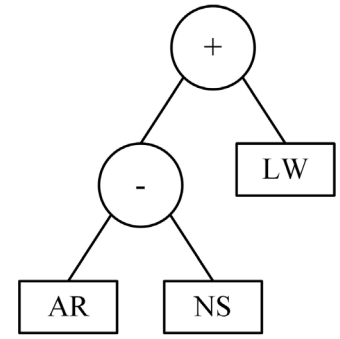


Fig. 3. A priority rule represented by a tree.

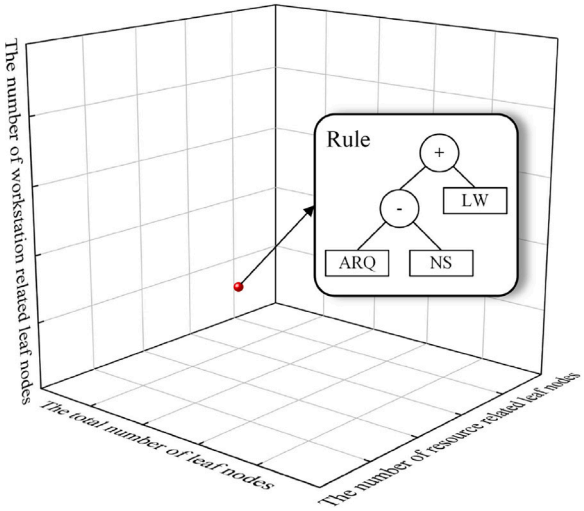


Fig. 4. A diagram of the feature map.

4.1.2. Searching the best rule

Since there exist quite a large number of rules with the combination of task attributes and arithmetic operators, this section develops a ME based hyper-heuristic method to search for the best rule. The ME performs the search based on a discretized multi-dimensional feature map.

To construct the feature map, three features are chosen: (1) the total number of leaf nodes in the rule; (2) the number of resource related leaf nodes (including the attributes RF, AR, MAR and MIR) in the rule; (3) the number of workstation related leaf nodes (including the attributes EW, LW and SW) in the rule. Each dimension of feature is discretized based on its range. A discretized feature map is then constructed consisting of many cells. Each cell points to a combination of feature values, and stores the rules with the corresponding feature values. The feature map is illustrated in Fig. 4.

Once a rule is generated, its feature values are calculated and the rule is put into the corresponding cell in the feature map. If a cell contains more than one rule, the rule with the lowest fitness value is retained. The fitness value of each rule is evaluated on a predefined set of problem instances called training set. In our study, the training set is created based on the data collected from the real AFAL (please refer to Section 5.2 for a detailed description for the training set). Assume that there are G instances in the training set. The fitness function is $\frac{1}{G}\sum_{g=1}^{G} F_g$, where F_g is the objective value (resource investment

cost) obtained by applying current rule to solve instance g based on the construction heuristics. The construction heuristics is a method of converting rules into task assignment decisions and evaluating their performances, whose details are introduced in the next Section.

The feature map is updated with the iteration of ME. Once the number of iterations reach the maximum value, the ME is terminated. To prevent over fitting, all the final individuals are evaluated on another set of instances called validation set, and the best individual on the validation set is selected as the final rule for that run of ME.

4.1.3. Construction heuristic algorithm

With the best rule obtained from the ME based hyper-heuristic method, a construction heuristic algorithm is developed to convert the rule into task assignment. Let U denote the set of unassigned tasks. The construction heuristics is presented as Algorithm 1. The assignment of tasks starts with the first workstation ($k = 1$). In lines 5–6, the next task to be assigned to the workstation is determined according to the priority rule. In lines 9–10, if the critical path length of current workstation is longer than the given cycle time, the task will be moved to the next workstation. After all the tasks are assigned to workstations, the objective function value of total resource investment cost is evaluated in line 13.

Algorithm 1 Construction heuristics of task assignment.

1: **Input:** A priority rule.
2: $k \leftarrow 1$; $U \leftarrow \{1, 2, \ldots, H-1\}$.
3: **while** $U \neq \emptyset$ **do**
4: Determine the assignable tasks from U following precedence relations.
5: Apply the rule to calculate the priority values of the assignable tasks.
6: Select the task with the smallest priority value and remove it from U.
7: Assign the selected task to workstation k, and calculate the critical path
8: length (CPL) of workstation k.
9: **if** CPL exceeds the given cycle time **then**
10: $k \leftarrow k+1$. Move the selected task to workstation k.
11: **end if**
12: **end while**
13: Evaluate the objective value.
14: **return** task assignment and objective value

To evaluate the objective function value given in Eq. (2), the starting times of tasks need to be obtained firstly. We have developed a simplified genetic algorithm to schedule the tasks in workstations, which enables us to calculate the objective function value accurately. The chromosome of GA is constructed from two parts. The first part is a task list that indicates a precedence feasible permutation of tasks. The second part is a resource capacity list where each element represents the invest capacity of a type of resource. Crossover and mutation are applied to modify the chromosome. To decode the chromosome into a solution with the starting time of each task, the serial schedule generating scheme is adopted to transform the task list into a schedule with the constraint imposed by the capacity list. After that, the resource investment cost and the smoothness index are evaluated. For the sake of brevity, we refer the readers to Shadrokh and Kianfar (2007) for the details of GA.

Algorithm 1 is essentially a single-pass heuristics. To improve the quality of initial task assignment, a multi-pass version of construction heuristics is developed. Different rules obtained in different runs of ME are input to Algorithm 1. Several task assignment results are thus constructed, the best of which is selected as the output of the first stage (i.e., the initial task assignment solution).

4.2. Task assignment improvement

An improvement heuristic (IH) algorithm is developed to improve the task assignment obtained in the first stage. The IH is designed based on the variable neighborhood search (VNS) algorithm. The neighborhoods of task assignment are firstly described, followed by the procedure of the IH algorithm.

4.2.1. Neighborhoods of task assignment solution

The neighborhoods of task assignment solution are created by modifying the solution with certain local search operators. To avoid obtaining an infeasible solution, three local search operators are applied for two adjacent workstations k and $k+1$.

The *move-1* operator tries to move one task from workstation k to $k+1$. The *move-2* operator moves one task from workstation $k+1$ to k. The *swap* operator exchanges two tasks that are respectively in workstation k and workstation $k+1$. Note that precedence constraints and cycle time constraints are satisfied in these operators. And at most L_1 attempts are performed to obtain a feasible solution for each operator. Fig. 5 illustrates the three operators, where the task assignment in Fig. 1 is taken as the example.

A total number of Z neighborhoods are generated based on the three operators. For instance, Z equals to six for the problem with three workstations.

4.2.2. Improvement heuristic algorithm

Let z be the serial number of neighborhoods, $z = 1, 2, \ldots, Z$. The procedure of the improvement heuristics is presented in Algorithm 2. L_2 iterations are executed for each neighborhood z. In each iteration, a task assignment solution is randomly chosen in the neighborhood, followed by the evaluation of the objective value. In lines 7–10, if a better solution is found, the task assignment is updated. If a better assignment is not found in L_2 iterations, the procedure continues with the next neighborhood ($z = z+1$). The procedure terminates if z reaches the maximum number of neighborhoods (Z). Once the procedure is terminated, the solution along with the best objective value are returned.

5. Computational analyses

Computational experiments and a real case study were conducted to evaluate the performance of our CITH approach. All the algorithms have been coded in Python 3.9 on a computer with an Intel Core i5-1135G7 2.40 GHz processor and 16 GB RAM. GUROBI 9.5 with parallel computing was used as the solver for mathematical models.

5.1. Parameter settings of instance

The parameters of instances are set based on the characteristics of the addressed AFAL. As depicted in Fig. 6, the AFAL is a paced moving assembly line consisting of three workstations.

With the data we have collected, the instance parameters are set as shown in Table 3. The parameters include the number of non-dummy tasks ($|J|-2$), the number of workstations ($|K|$), resource requirement (w_{jr}) and task processing time (p_j), the number of resource types ($|R|$), and the unit cost of each type of resource (c_r). w_{jr} takes the value in the range $[0, 6]$, where 0 means that task j has no requirements for resource r. p_j ranges from one hour to eight hours. c_r adds a cost weight for each type of resource, which is assumed to vary in the range $[1, 5]$. The cycle time is calculated as $T = 1.3EF_H^{max}/|K|$, where EF_H^{max} is the earliest estimated finishing time of the end dummy task in the predefined precedence graph.

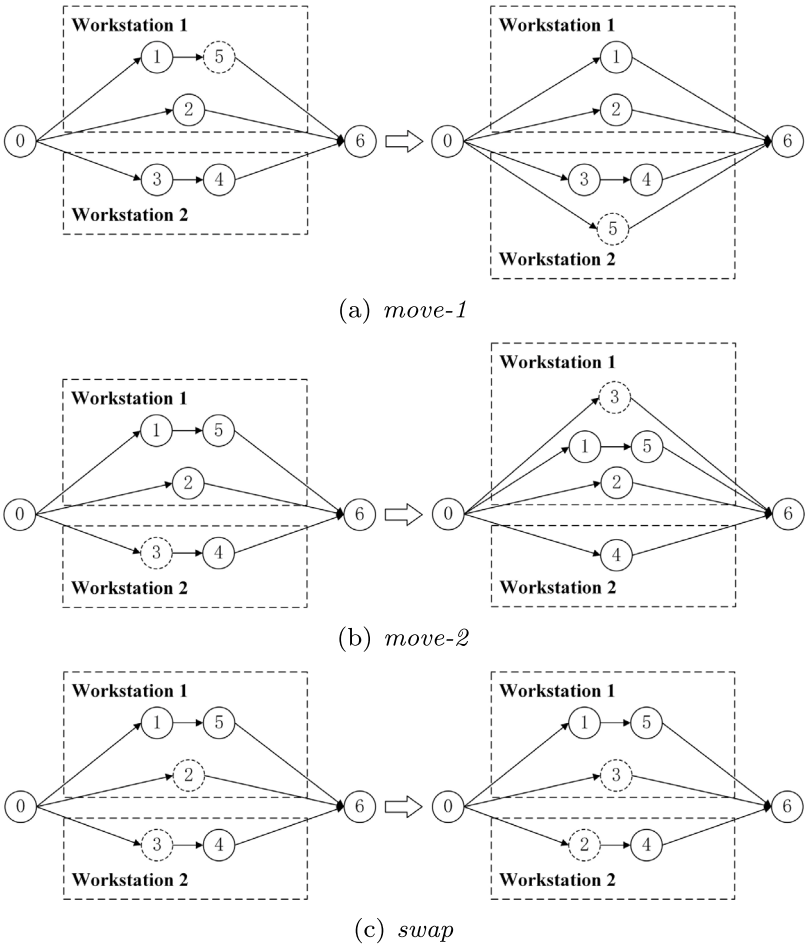


Fig. 5. An illustration of the local search operators in two adjacent workstations.

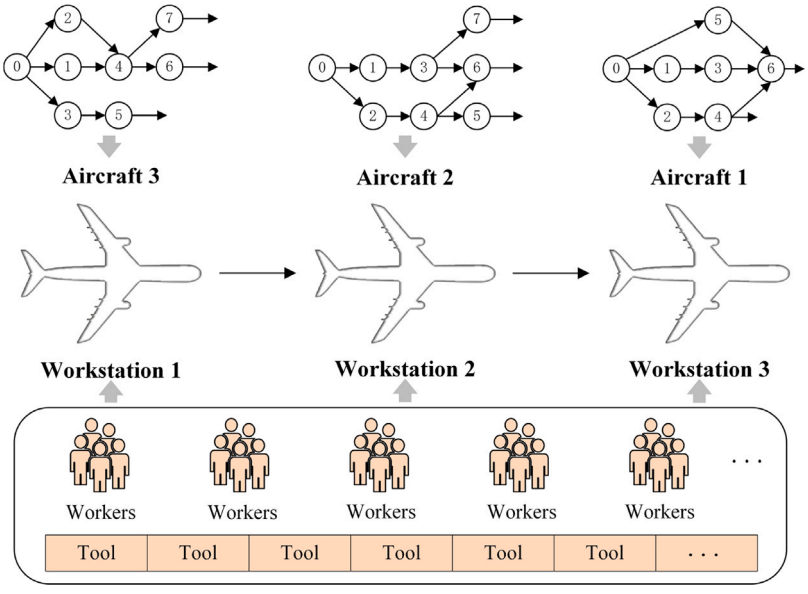


Fig. 6. A graph representation of the aircraft final assembly line.

5.2. Performance of the task assignment rules

To train and test the task assignment rules, one training set, one validation set and four testing sets are generated randomly based on the parameter settings in Table 3. The training set contains one hundred instances with 20 non-dummy tasks. The validation set consists of instances with four different sizes (the number of non-dummy tasks are respectively 20, 30, 60 and 120) and there are ten instances for each size. The purpose of having the validation set is to improve the generalization ability of the evolved rules. The number of non-dummy

Algorithm 2 The improvement heuristic algorithm.

1: **Input:** An initial task assignment $\boldsymbol{x}$ and its objective vale F.
2: $z \leftarrow 1$.
3: **while** $z \leq Z$ **do**
4: **for** $l = 1$ **to** L_2 **do**
5: $\boldsymbol{x}' \leftarrow$ Select a task assignment solution in neighborhood z.
6: Evaluate the objective function value F' of solution $\boldsymbol{x}'$.
7: **if** $F' < F$ **then**
8: $F \leftarrow F'$; $\boldsymbol{x} \leftarrow \boldsymbol{x}'$; $z \leftarrow 1$.
9: Go to line 4.
10: **end if**
11: **end for**
12: $z \leftarrow z + 1$.
13: **end while**
14: **return** $\boldsymbol{x}$ and F

Table 3
Parameter settings of random instances.

Parameter	Value
$\|J\| - 2$	$\{20, 30, 60, 120\}$
$\|K\|$	3
w_{jr}	$[0, 6]$
p_j	$[1, 8]$
$\|R\|$	4
c_r	$[1, 5]$

Table 4
Parameters values of the ME.

Parameter	Value
Number of iterations	30
Maximum depth of tree	5
Crossover probability	0.8
Mutation probability	0.2
Range of feature 1	$[1, 31]$
Range of feature 2	$[0, 12]$
Range of feature 3	$[0, 12]$
Feature discretization value	$(6, 6, 6)$

tasks in the four testing sets is 20, 30, 60, and 120, respectively. And for each testing set, there are fifty instances.

With the training set and validation set, the ME is performed with the parameter settings in Table 4. The discretization value of each feature is six, so the feature map includes $6 \times 6 \times 6 = 216$ cells, meaning that the archive can store at most 216 different rules.

To evaluate the performance of the task assignment rule evolved by ME, the GP (genetic programming) rule (Braune et al., 2022) and eight classic rules (Kellegöz & Toklu, 2015) are used to compare with the ME rule. The eight classic rules are: GPW (Greatest positional weight), MPL (Maximum processing time over latest workstation), MEW (Minimum earliest workstation), MLW (Minimum latest workstation), LPT (Longest processing time), MTS (Maximum number of total successors), GCW (Greatest cumulative workload) and GCRR (Greatest cumulative resource requirement). All the rules are applied to solve the instances in the test sets following the procedure of Algorithm 1. Each rule is executed five times for each test instance and the resulting best objective value is adopted for the comparison. The relative percentage deviation (RPD) defined as Eq. (16) is used to measure the performance of the rules.

$$RPD = \frac{C(A) - C^*}{C(A)} \times 100\% \tag{16}$$

where $C(A)$ is the objective value of rule A, and the C^* is the best objective value of all the rules. The smaller the RPD, the better the rule.

Table 5
Summary of the average RPDs of the compared rules.

Testing set	GRW	MPL	MEW	MLW	LPT	MTS	GCW	GCRR	GP	ME
20 tasks	2.51	3.63	3.07	2.53	3.66	2.43	3.70	4.08	1.59	1.11
30 tasks	2.38	3.53	3.83	2.47	4.21	2.14	4.43	4.47	1.59	1.22
60 tasks	1.81	3.64	3.30	1.76	3.84	2.05	4.66	4.29	1.23	0.88
120 tasks	2.54	4.51	3.67	2.01	4.78	2.19	4.83	5.54	1.27	0.53
Average	2.31	3.83	3.47	2.19	4.12	2.20	4.41	4.60	1.42	0.94

Table 5 reports the average RPD (in percentage) of each rule on each testing set. The ME has the lowest average RPD of all the testing sets, followed by the GP. It implies that the rules generated by ME can construct the task assignment that is superior to those of the GP rule and the classic rules. Moreover, the average RPD of the ME rule on the instances with 60 tasks and 120 tasks are below 1%, indicating that the superiority of the ME rule over other rules appears to be more apparent for large instances.

The above results demonstrate that the ME based hyper-heuristic can discover fine and implicit knowledge for balancing the aircraft final assembly line. In the following, three best-performing rules that have the lowest average objective values on the validation set are selected as the input to the construction of task assignment. Fig. 7 depicts the three selected best rules.

5.3. Performance of the CITH approach

To test the proposed CITH approach, four different sets of instances were generated. The number of non-dummy tasks in each set is 20, 30, 60 and 120, respectively. Each set includes fifty random instances.

A set of experiments was carried out preliminarily to tune the parameters of the CITH. The number of attempts (L_1) for each neighborhood operator is set to three and the number of iterations (L_2) of the IH algorithm is set to two.

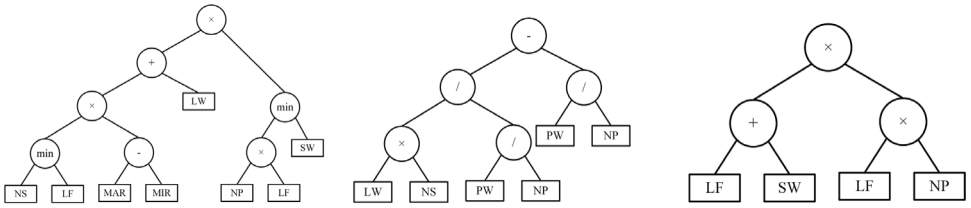
5.3.1. Comparison with GUROBI on small instances

Firstly, the CITH was compared with GUROBI solver in this section. Small instances with 20 and 30 non-dummy tasks were solved. For each instance, the time limit of GUROBI was set as 1800 s, and the CITH was executed ten times.

Table 6 reports the comparison results. For each instance set, average objective value obtained by GUROBI (F_G) and CITH (F_S), the average CPU time (in seconds) of GUROBI (t_G) and CITH (t_S) are respectively reported. #opt means the number of instances that were solved to optimum by GUROBI. F_S^{best} refers to the average of the best objective value obtained in ten runs of CITH. In the last two columns, average gaps between the two methods are also reported.

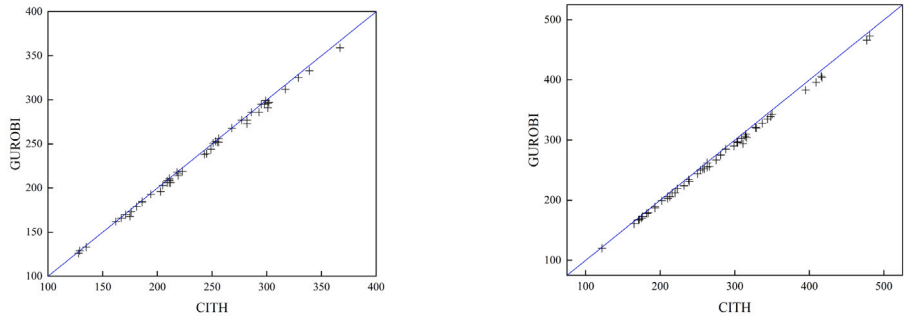
All the instances with 20 tasks were solved optimally by GUROBI. However, for the instance set with 30 tasks, only 19 out of 50 instances were solved optimally. Average CPU time of GUROBI exhibits a dramatical increase. On the contrary, average computational time of our CITH does not exceed 26 s for both sets of instances. The average gaps between GUROBI and CITH are less than 3.8%. It suggests that the proposed approach can obtain near optimal solutions in reasonable times for small instances.

Fig. 8 displays the comparison of all the objective values obtained by the two methods. The diagonal line represents that both methods have equal results, and points "+" below the line mean the objective



(a) rule #1 (b) rule #2 (c) rule #3

Fig. 7. The task assignment rules found by ME.



(a) 20 tasks (b) 30 tasks

Fig. 8. Overview of the objective values of CITH and GUROBI.

Table 6
Comparison results of GUROBI and CITH on small instances.

Instance	GUROBI			CITH			Gap between F_G and F_S (%)	Gap between F_G and F_S^{best} (%)
	F_G	t_G(s)	#opt	F_S	F_S^{best}	t_S(s)		
20-task	232.30	19.9	50	238.37	235.42	14.54	2.51	1.30
30-task	267.00	1367.15	19	280.40	273.88	25.37	3.74	2.49
Average	249.65	693.53	34.50	259.39	254.65	19.96	3.13	1.90

values of CITH are larger than that of GUROBI. It can be found that the points are very close to the diagonal line for both sets of instances, which further demonstrates that our CITH can generate solutions with high accuracy for small instances.

5.3.2. Comparison with heuristic algorithms

For solving medium and large instances, our CITH are compared with the following three heuristic algorithms: (1) The particle swarm optimization-based heuristic algorithm (denoted as PSO) proposed by Şahin and Kellegöz (2019) to solve the resource investment and balancing problem; (2) The variant of CITH (designated as CITH_1) that omits the first stage; (3) The variant of CITH (referred to as CITH_2) that does not include the second stage. Instances with 30 tasks, 60 tasks and 120 tasks were solved. For each instance, each heuristic was run ten times. Table 7 summarizes the results. The average RPD (in percentage) and average CPU time (in seconds) of all the compared algorithms are reported.

From Table 7, it is observed that:

(1) The CITH and its variants outperform the existing approach PSO since they have lower average RPDs.
(2) The average RPD of CITH is zero and lower than that of CITH_1 and CITH_2. Therefore, the CITH performs the best, followed by CITH_2 and CITH_1.

Table 7
Summary of the results obtained by the compared heuristic algorithms.

Instance	Average RPD				Average CPU time			
	PSO	CITH_1	CITH_2	CITH	PSO	CITH_1	CITH_2	CITH
30 tasks	14.94	2.38	1.05	0	11.88	17.71	8.91	25.37
60 tasks	21.35	2.64	1.14	0	82.54	79.56	28.43	114.33
120 tasks	23.73	4.32	1.12	0	450.69	319.41	134.11	437.39
Average	20.01	3.11	1.10	0	181.70	138.89	57.15	192.36

(3) The average RPD of CITH_1 is 3.11%. It shows that the construction heuristics based on rules can enhance the algorithm performance.
(4) The average RPD of CITH_2 is 1.10%. It shows that the task assignment improvement strategy based on IH can further improve the solution quality.

The gap between CITH and CITH_2 is relatively small. That is because the first stage of our approach has provided solutions with quite good quality. Nevertheless, compared to CITH_2, CITH improves RPD by over 1%. The average CPU time of CITH is the longest among the compared algorithms but considered acceptable with about seven minutes for 120-task instances.

To capture a broad overview of the differences between the proposed CITH and its variants, Fig. 9 displays the boxplots of RPDs of

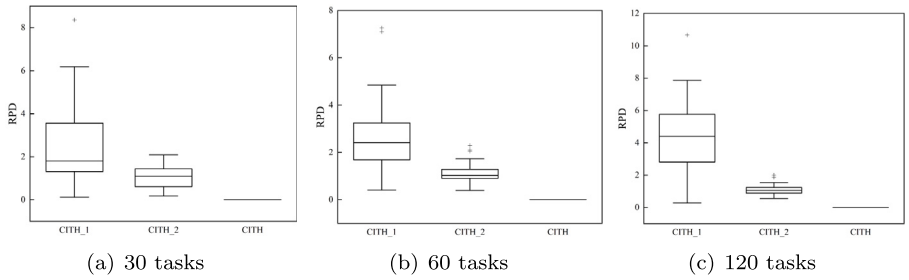


Fig. 9. Boxplots of the RPDs for all the compared algorithms.

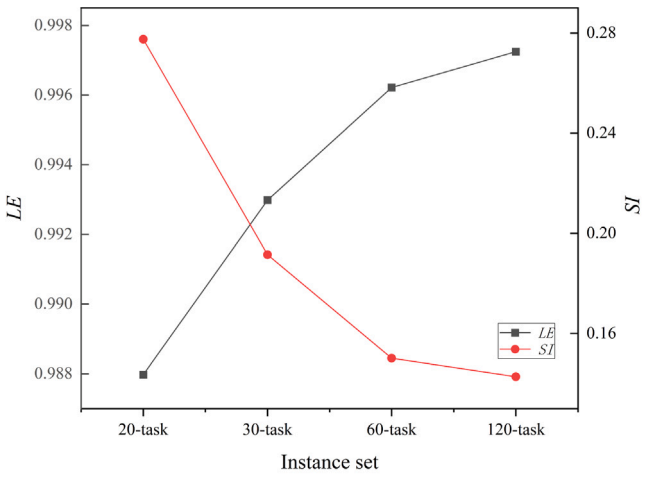


Fig. 10. The balancing metrics for all the sets of instances.

Table 8
Results of pair-sample Wilcoxon-test on the compared heuristics.

Compared heuristics	30 tasks	60 tasks	120 tasks
(PSO, CITH_1)	<0.001	<0.001	<0.001
(PSO, CITH_2)	<0.001	<0.001	<0.001
(PSO, CITH)	<0.001	<0.001	<0.001
(CITH_1, CITH_2)	<0.001	<0.001	<0.001
(CITH_1, CITH)	<0.001	<0.001	<0.001
(CITH_2, CITH)	<0.001	<0.001	<0.001

CITH variants for each instance set. For all the instances, PRDs of CITH are zero, but RPDs of other variants are larger than zero. The algorithms (CITH and CITH_2) that use the rule-based construction heuristic have a lower median and 25th (75th) percentile of RPD. The result further confirms the effectiveness of the proposed approach.

The pair-sample Wilcoxon-test was conducted to evaluate the statistical differences between the compared heuristics in objective value. The result of pair-sample Wilcoxon-test is summarized in Table 8, in which data on columns 2–4 are p-values for the instance sets with 30, 60 and 120 tasks. For each set of instances, there is significant difference between any two compared heuristics.

5.4. Evaluation of the objective function values

Proposition 1 allows us to optimize solely on the resource investment cost. In this section, we evaluate the performance of our CITH approach on the other objective function, SI. In addition, line efficiency (LE) is also used to assess the line balancing. LE is calculated as below.

$$LE = \frac{\sum_{k \in K} T_k}{|K|C} \tag{17}$$

The value of SI is bigger than zero and a lower SI value implies a better balancing line. The value of LE varies in the range $[0, 1]$. A large LE being close to one means a high line efficiency. Four instance sets with fifty instances in each set were solved by our CITH. Fig. 10 depicts average value of the metrics for each instance set.

For all the instance sets, LE is larger than 0.98 (close to one) and SI is lower than 0.28. It shows that our approach has a pretty good performance in line balancing. As the number of tasks rises, LE increases and SI decreases. It implies that for larger instances, the effectiveness of our approach is more evident.

Table 9
Comparison of the CITH solution and the real assembly line.

Method	Resource investment cost	SI	LE	Resource capacity	Actual cycle time
Our CTIH	318	0.00	1.00	{43, 22, 20, 21}	{46, 46, 46}
Real assembly line	420	3.51	0.95	{54, 30, 27, 29}	{40, 45, 46}

5.5. Study on a real case

In the addressed AFAL, 166 assembly tasks need to be assigned to three workstations. The tasks are mainly the installation and testing of electrical wiring interconnection system, avionics system and aircraft interior. The given cycle time of each workstation is forty-six hours. We present the detailed information of the AFAL in Appendix B, including processing times and resource requirements of assembly tasks, as well as precedence relations.

We compare the line balancing solution obtained from our CITH with the real assembly line. In the workshop, the GPW rule was used for task assignment and line balancing. Table 9 summarizes the CITH solution and the real data. Data including resource investment cost, smoothness index (SI), line efficiency (LE), capacity of each type of resource and actual cycle time of each workstation are reported. Appendix C shows the detailed information of both solutions.

As compared with the real assembly line, our CITH has a quite different task assignment solution and reduces the resource investment cost by 24.29%. According to the values of SI and LE, our CITH achieved a more balanced effect with each workstation having exactly the same cycle time.

The above result shows that different task assignment solutions may result in quite different resource investment strategies. To further evaluate the impact of task assignment on resource investment, we randomly modify the CITH solution by performing moving operators or swap operator for thirty times. The corresponding resource investment costs of the thirty modifications are depicted in Fig. 11. The blue line represents the resource investment cost of the CITH solution. All

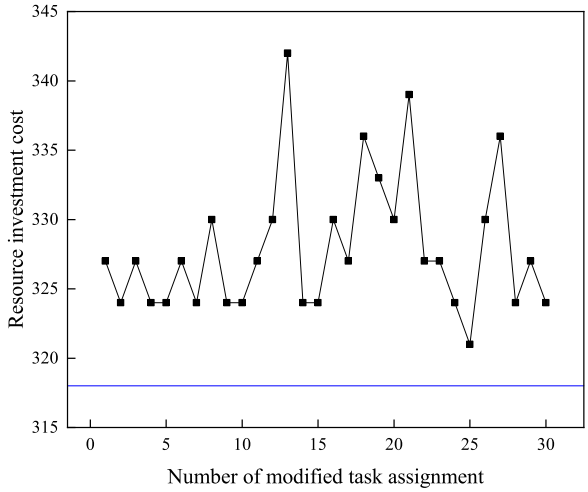


Fig. 11. Resource investment costs of the thirty modifications on task assignment.

the modifications lead to an increase of the cost with the maximum being up to more than 4% . It implies that resource investment is very sensitive to task assignment decision. Therefore, the resource constraints cannot be ignored in the balancing of aircraft assembly.

6. Conclusions and future work

This paper studies the aircraft final assembly balancing problem considering resource constraints (AALBP-RC). An integer linear programming model is formulated. A construction and improvement based two-stage heuristic approach is developed to solve the problem. In the first stage, the initial task assignment is constructed based on the multi-pass of the rules evolved by ME. In the second stage, an improvement heuristic algorithm with customized operators is developed to improve the task assignment. Moreover, a GA is inserted for task scheduling to evaluate the actual cycle time and resource capacity.

Three high-performing task assignment rules with fine knowledge were found by ME and incorporated into the CITH. The comparative experiments demonstrate the superiority of our CITH in terms of reducing resource investment cost. Both the first stage construction heuristic and the second stage improvement heuristic play important roles in enhancing the algorithm performance. In addition, a real case study revealed the effectiveness of our CITH in solving practical problem.

Unlike machines, there exist multiple skills or multiple modes (i.e., the combination of worker requirement and task processing time) for workers. Different selections of skills or modes may result in different processing costs or times. Therefore, incorporating human resource allocation into the aircraft assembly line balancing problem is worth being studied in the future. On the other hand, as the production batch of China's commercial aircraft increases, learning effect emerges. The cycle time must be adjusted to achieve the target production capacity. Thus, dynamic balancing of the aircraft assembly line to guarantee smooth batch production could also be a valuable extension of this study.

CRediT authorship contribution statement

Zhongkai Bao: Conceptualization, Methodology, Software, Writing. **Lu Chen:** Conceptualization, Writing, Funding acquisition. **Kejun Qiu:** Data curation, Software, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 52175475). The authors thank the referees and the editors for their valuable comments.

Appendix A. Calculating the earliest and latest workstation of a task

The earliest workstation (e_j) and latest workstation (l_j) that a task (j) can be assigned to are two important task attributes. The e_j and l_j calculated for classical ALBPs (Scholl & Becker, 2006) cannot be directly applied to our AALBP-RC, since there exist parallel tasks at workstations in the AALBP-RC. We thus use the following procedure to determine the e_j and l_j.

Step 1: Define $A_1 = \{1, 2, \ldots, H-1\}$, $E_1 = \{0\}$, $A_2 = \{1, 2, \ldots, H-1\}$, and $E_2 = \{H\}$. Assume the earliest workstation of task zero is $e_0 = 1$ and the latest workstation of task H is $l_H = |K|$.

Step 2: $G_1 = \emptyset$. $G_2 = \emptyset$. Put the tasks whose all immediate predecessors are in E_1 into G_1. Put the tasks whose all immediate successors are in E_2 into G_2.

Step 3: For each task j in G_1, the maximum of the earliest workstation that its all predecessors can be assigned to is denoted as e_j^c. If the length of the critical path created by task j and its predecessors with the earliest workstation be e_j^c is smaller than the given cycle time, the earliest workstation of task j is $e_j = e_j^c$; otherwise $e_j = e_j^c + 1$.

For each task j in G_2, the minimum of the latest workstation that its all successors can be assigned to is denoted as l_j^c. If the length of the critical path created by task j and its successors with the latest workstation be l_j^c is smaller than the given cycle time, the latest workstation of task j is $l_j = l_j^c$; otherwise $l_j = l_j^c - 1$.

Step 4: Put the tasks in G_1 (G_2) into E_1 (E_2) and delete these tasks from A_1 (A_2). If $A_1 = \emptyset$ and $A_2 = \emptyset$, the procedure terminates; otherwise returning to step 2.

With the above procedure, e_j and l_j are obtained and the slack workstation of a task (j) is calculated as: $l_j - e_j + 1$.

Appendix B. The information of real case

The information of real case is presented in Table B.1. The "Task" column gives the serial number of tasks. The "PT" refers to processing time (in hour). Requirements of worker resource and tool resources are reported. The predecessors are the tasks that must be completed before current task is started. For example, task 3 cannot be started before tasks 1 and 2 are completed. Additionally, the resource unit cost is assumed to be three.

Appendix C. The task assignment solutions of real case

Table C.1 presents the task assignment solution obtained by our CITH. Table C.2 presents the task assignment solution in real assembly line. The tasks assigned to each workstation are reported.

Table B.1
The information of real case.

Task	PT	Worker	Tool 1	Tool 2	Tool 3	Predecessors	Task	PT	Worker	Tool 1	Tool 2	Tool 3	Predecessors
1	3	2	1	1	3	0	84	4	2	1	1	1	74, 81, 82
2	3	2	0	0	1	0	85	1	2	0	0	0	83, 84
3	2	2	1	0	1	1, 2	86	16	1	1	2	1	0
4	2	2	0	1	1	1, 2	87	8	1	1	0	3	0
5	4	1	1	0	2	1, 2	88	16	1	1	3	1	0
6	4	1	1	3	0	3, 4	89	16	1	0	1	0	0
7	4	1	1	2	1	3, 4	90	12	1	0	1	1	0
8	4	2	1	1	2	5	91	16	1	1	0	1	0
9	4	2	0	1	1	5	92	12	1	2	0	2	0
10	2	2	0	1	0	6, 7	93	12	1	1	1	1	0
11	2	2	1	0	1	6, 7	94	12	1	0	1	1	0
12	12	2	1	1	2	8, 9, 10, 11	95	12	1	1	0	1	0
13	12	2	1	1	1	8, 9, 10, 11	96	8	1	1	0	0	87
14	4	2	0	2	0	8, 9, 10, 11	97	6	1	1	0	1	90, 92, 93, 94, 95
15	4	2	1	1	1	14	98	8	1	0	0	1	90, 92, 93, 94, 95
16	4	2	2	1	1	15	99	8	1	1	1	2	90, 92, 93, 94, 95
17	4	2	0	1	0	16	100	12	1	1	1	1	90, 92, 93, 94, 95
18	3	2	2	0	1	12, 13, 17	101	8	1	1	1	1	90, 92, 93, 94, 95
19	3	2	1	2	1	12, 13, 17	102	8	1	2	2	1	86, 88, 89, 91, 96
20	4	2	1	1	1	12, 13, 17	103	8	1	0	3	1	86, 88, 89, 91, 96
21	8	3	1	1	1	18, 19	104	8	1	0	1	1	86, 88, 89, 91, 96
22	8	2	2	1	0	18, 19	105	8	1	2	2	2	86, 88, 89, 91, 96
23	2	2	2	3	1	20	106	4	1	0	0	1	86, 88, 89, 91, 96
24	4	2	2	1	0	20	107	4	1	1	0	0	97
25	2	2	2	1	1	23	108	4	1	2	1	1	98, 99, 101, 106
26	4	2	1	1	1	24, 25	109	4	1	1	2	0	98, 99, 101, 106
27	2	2	1	1	1	21, 22	110	15	2	1	0	1	100, 102, 103, 104, 105, 108, 109
28	2	2	1	1	2	21, 22	111	18	2	0	1	0	100, 102, 103, 104, 105, 108, 109
29	4	2	1	1	1	21, 22	112	13	2	2	0	0	100, 102, 103, 104, 105, 107, 108, 109
30	5	2	0	0	0	26, 27, 28	113	12	2	2	1	0	100, 102, 103, 104, 105, 108, 109
31	5	2	1	1	0	26, 27, 28	114	12	2	0	0	0	100, 102, 103, 104, 105, 108, 109
32	8	2	3	0	0	29, 30, 31	115	16	2	1	2	0	113, 114
33	8	2	1	1	1	29, 30, 31	116	12	2	0	1	0	113, 114
34	4	2	1	0	2	29, 30, 31	117	15	2	0	1	1	112
35	4	2	1	1	1	29, 30, 31	118	15	2	1	0	1	110
36	2	1	1	1	1	29, 30, 31	119	10	2	0	0	1	111
37	2	1	1	0	0	36	120	11	2	1	1	1	116
38	1	1	0	1	0	37	121	8	2	1	0	0	115, 117, 119,
39	1	1	1	1	1	38	122	8	2	0	1	1	115, 117, 119,
40	4	2	1	0	3	34, 35, 39	123	13	2	0	2	2	115, 117, 119,
41	1	1	1	0	1	34, 35, 39	124	19	2	1	1	0	118
42	1	1	1	0	0	41	125	8	1	1	1	0	120
43	1	1	0	1	1	42	126	8	1	3	0	1	120
44	1	1	0	1	1	43	127	4	1	1	1	0	121, 122
45	16	3	0	1	1	32, 33, 40, 44	128	4	1	0	1	1	121, 122
46	4	2	1	1	1	32, 33, 40, 44	129	7	2	1	2	1	121, 122
47	1	1	0	1	1	32, 33, 40, 44	130	9	2	0	0	1	127, 128
48	4	2	2	1	1	47	131	8	1	1	0	3	123
49	4	2	0	0	2	46	132	8	1	1	0	2	123
50	2	2	0	1	0	48	133	5	1	1	0	1	125, 126, 129
51	2	2	1	0	0	50	134	8	1	1	2	1	125, 126, 129
52	4	2	1	1	0	49	135	8	1	1	1	1	125, 126, 129
53	2	2	0	1	0	51	136	12	2	1	1	0	124, 130, 131, 132
54	4	2	2	1	0	52, 53	137	8	2	1	0	1	124, 130, 131, 132
55	4	2	1	1	2	52, 53	138	6	2	1	1	2	124, 130, 131, 132, 133
56	2	2	1	0	2	45, 54, 55	139	8	1	1	0	1	124, 130, 131, 132
57	8	2	1	0	1	45, 54, 55	140	8	1	1	1	1	124, 130, 131, 132
58	4	2	1	0	1	45, 54, 55	141	8	1	1	1	1	134, 135
59	5	2	1	0	0	45, 54, 55	142	8	1	2	0	0	134, 135
60	4	2	2	1	1	56	143	8	5	1	1	0	0
61	1	1	0	3	0	59	144	8	5	1	1	0	143
62	4	2	1	1	1	60, 61	145	4	3	1	1	1	144
63	3	1	1	1	0	58, 60, 61	146	4	3	0	1	1	144
64	3	1	1	1	0	58, 60, 61	147	2	2	1	0	1	145, 146
65	1	1	3	1	1	60, 61	148	3	2	0	1	1	147
66	1	1	1	0	1	63, 64, 65	149	3	3	0	0	1	145, 146
67	1	1	1	1	1	63, 64, 65	150	3	2	1	1	1	148, 149
68	1	2	1	1	1	57, 66, 67	151	3	2	0	0	2	148, 149
69	1	1	1	1	0	57, 66, 67	152	8	3	1	1	1	150, 151

(*continued on next page*)

Table B.1 (*continued*).

70	1	1	1	1	2	57, 66, 67	153	8	3	1	0	0	150, 151
71	4	2	1	1	1	68, 69, 70	154	8	3	1	1	0	152, 153
72	1	1	1	1	1	68, 69, 70	155	8	6	1	0	1	152, 153
73	1	1	1	3	0	68, 69, 70	156	12	4	0	1	0	154, 155
74	5	2	1	0	2	62, 72, 73	157	12	4	2	1	1	154, 155
75	5	1	0	1	0	62, 72, 73	158	4	2	0	2	3	156, 157
76	5	1	0	0	1	62, 72, 73	159	4	2	0	1	0	156, 157
77	4	1	2	1	1	75, 76	160	4	2	1	1	2	158, 159
78	4	1	3	2	1	75, 76	161	4	2	1	2	3	158, 159
79	4	1	1	0	1	77, 78	162	8	3	0	0	2	160, 161
80	4	1	1	1	0	77, 78	163	8	3	0	2	1	160, 161
81	4	2	1	0	1	71, 79	164	8	3	0	1	0	162, 163
82	4	1	1	2	0	71, 79	165	4	3	2	1	1	162, 163
83	4	2	1	0	0	74, 81, 82	166	2	3	1	1	0	165

Table C.1
Task assignment obtained by our CITH.

Workstation 1:
{1, 2, 5, 3, 4, 8, 9, 143, 6, 7, 87, 86, 88, 89, 91, 10, 11, 90, 92, 93, 94, 95, 144, 14, 12, 13, 96, 15, 16, 145, 146, 98, 99, 101, 17, 100, 106, 102, 103, 104, 105, 97, 107, 108, 109, 18, 19, 147, 20, 149, 21, 22, 23, 25, 148, 151}
Workstation 2:
{24, 26, 27, 28, 29, 30, 31, 113, 114, 150, 36, 34, 35, 111, 37, 32, 33, 38, 112, 39, 152, 153, 115, 117, 119, 40, 41, 42, 43, 44, 154, 155, 46, 47, 48, 45, 49, 50, 121, 122, 52, 51, 53, 156, 157, 54, 55, 116, 110, 120, 118, 56, 59, 158, 159, 60, 61, 58}
Workstation 3:
{123, 57, 124, 63, 64, 125, 126, 65, 160, 161, 127, 128, 66, 67, 129, 133, 68, 69, 70, 62, 131, 132, 130, 136, 138, 137, 139, 140, 72, 73, 162, 163, 165, 166, 164, 75, 76, 77, 78, 80, 134, 135, 141, 142, 71, 79, 74, 81, 82, 83, 84, 85}

Table C.2
Task assignment in real assembly line.

Workstation 1:
{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 14, 12, 13, 15, 16, 17, 18, 19, 113, 114, 20, 21, 22, 23, 24, 25, 26, 27, 28, 97, 90, 92, 93, 94, 95, 86, 87, 88, 89, 91, 96, 98, 99, 101, 100, 106, 102, 103, 104, 105, 108, 109}
Workstation 2:
{30, 31, 29, 107, 111, 112, 115, 117, 119, 36, 34, 35, 37, 38, 39, 32, 33, 40, 41, 42, 43, 44, 143, 144, 145, 146, 46, 47, 147, 48, 121, 122, 149, 148, 45, 49, 50, 52, 51, 150, 151, 53, 54, 55, 152, 153, 110, 116}
Workstation 3:
{154, 155, 56, 59, 118, 60, 123, 61, 120, 58, 57, 63, 64, 124, 65, 156, 157, 66, 67, 127, 128, 68, 69, 70, 62, 125, 126, 130, 129, 131, 132, 72, 73, 158, 159, 75, 76, 160, 161, 77, 78, 134, 135, 162, 163, 71, 79, 74, 81, 82, 136, 133, 137, 139, 140, 141, 142, 164, 138, 165, 83, 84, 80, 166, 85}

References

Abdinnour, S. (2011). Hawker Beechcraft uses a new solution approach to balance assembly lines. *Interfaces*, *41*(2), 164–176.

Ağpak, K., & Gökçen, H. (2005). Assembly line balancing: Two resource constrained cases. *International Journal of Production Economics*, *96*(1), 129–140.

Alakaş, H. M. (2021). General resource-constrained assembly line balancing problem: conjunction normal form based constraint programming models. *Soft Computing*, *25*(8), 6101–6111.

Arkhipov, D., Battaïa, O., Cegarra, J., & Lazarev, A. (2018). Operator assignment problem in aircraft assembly lines: a new planning approach taking into account economic and ergonomic constraints. *Procedia CIRP*, *76*, 63–66.

Battaïa, O., & Dolgui, A. (2013). A taxonomy of line balancing problems and their solutionapproaches. *International Journal of Production Economics*, *142*(2), 259–277.

Borreguero, T., Mas, F., Menéndez, J. L., & Barreda, M. (2015). Enhanced assembly line balancing and scheduling methodology for the aeronautical industry. *Procedia Engineering*, *132*, 990–997.

Boysen, N., Fliedner, M., & Scholl, A. (2007). A classification of assembly line balancing problems. *European Journal of Operational Research*, *183*(2), 674–693.

Braune, R., Benda, F., Doerner, K. F., & Hartl, R. F. (2022). A genetic programming learning approach to generate dispatching rules for flexible shop scheduling problems. *International Journal of Production Economics*, *243*, Article 108342.

Caggiano, A., Marzano, A., & Teti, R. (2016). Resource efficient configuration of an aircraft assembly line. *Procedia CIRP*, *41*, 236–241.

Chand, S., Rajesh, K., & Chandra, R. (2022). MAP-elites based hyper-heuristic for the resource constrained project scheduling problem. arXiv preprint arXiv:2204.11162.

Chen, Y.-Y., Cheng, C.-Y., & Li, J.-Y. (2018). Resource-constrained assembly line balancing problems with multi-manned workstations. *Journal of Manufacturing Systems*, *48*, 107–119.

Corominas, A., Ferrer, L., & Pastor, R. (2011). Assembly line balancing: general resource-constrained case. *International Journal of Production Research*, *49*(12), 3527–3542.

Eghtesadifard, M., Khalifeh, M., & Khorram, M. (2020). A systematic review of research themes and hot topics in assembly line balancing through the web of science within 1990–2017. *Computers & Industrial Engineering*, *139*, Article 106182.

Eslamipoor, R., & Nobari, A. (2021). A mathematical model for an integrated assembly line regarding learning and fatigue effects. *Robotica*, *39*(8), 1434–1450.

Fang, Y., Xu, H., Liu, Q., & Pham, D. T. (2020). Evolutionary optimization using epsilon method for resource-constrained multi-robotic disassembly line balancing. *Journal of Manufacturing Systems*, *56*, 392–413.

Fang, P., Yang, J., Liao, Q., Zhong, R. Y., & Jiang, Y. (2021). Flexible worker allocation in aircraft final assembly line using multiobjective evolutionary algorithms. *IEEE Transactions on Industrial Informatics*, *17*(11), 7468–7478.

Jin, W., He, Z., & Wu, Q. (2022). Robust optimization of resource-constrained assembly line balancing problems with uncertain operation times. *Engineering Computations*, *39*(3), 813–836.

Kellegöz, T., & Toklu, B. (2015). A priority rule-based constructive heuristic and an improvement method for balancing assembly lines with parallel multi-manned workstations. *International Journal of Production Research*, *53*(3), 736–756.

Liu, R., Liu, M., Chu, F., Zheng, F., & Chu, C. (2021). Eco-friendly multi-skilled worker assignment and assembly line balancing problem. *Computers & Industrial Engineering*, *151*, Article 106944.

Lovato, D., Guillaume, R., Thierry, C., & Battaia, O. (2022). Managing disruptions in aircraft assembly lines with staircase criteria. *International Journal of Production Research*, 1–17.

Lu, Z., Ren, Y., Wang, L., & Zhu, H. (2019). A resource investment problem based on project splitting with time windows for aircraft moving assembly line. *Computers & Industrial Engineering*, *135*, 568–581.

Mouret, J.-B., & Clune, J. (2015). Illuminating search spaces by mapping elites. arXiv preprint arXiv:1504.04909.

Ogan, D., & Azizoglu, M. (2015). A branch and bound method for the line balancing problem in U-shaped assembly lines with equipment requirements. *Journal of Manufacturing Systems*, *36*, 46–54.

Ozdemir, R., Sarigol, I., AlMutairi, S., AlMeea, S., Murad, A., Naqi, A., & AlNasser, N. (2021). Fuzzy multi-objective model for assembly line balancing with ergonomic risks consideration. *International Journal of Production Economics*, *239*, Article 108188.

Pinhão, J. d. M. O. B., Ignacio, A. A. V., & Coelho, O. (2022). An integer programming mathematical model with line balancing and scheduling for standard work optimization: A realistic application to aircraft engines assembly lines. *Computers & Industrial Engineering*, *173*, Article 108652.

Pitakaso, R., Sethanan, K., Jirasirilerd, G., & Golinska-Dawson, P. (2021). A novel variable neighborhood strategy adaptive search for SALBP-2 problem with a limit on the number of machine's types. *Annals of Operations Research*, 1–25.

Quyen, N. T. P., Chen, J. C., & Yang, C.-L. (2017). Hybrid genetic algorithm to solve resource constrained assembly line balancing problem in footwear manufacturing. *Soft Computing*, *21*, 6279–6295.

Ren, Y., & Lu, Z. (2019). A flexible resource investment problem based on project splitting for aircraft moving assembly line. *Assembly Automation*, *39*(4), 532–547.

Ren, Y., Lu, Z., & Liu, X. (2020). A branch-and-bound embedded genetic algorithm for resource-constrained project scheduling problem with resource transfer time of aircraft moving assembly line. *Optimization Letters*, *14*(8), 2161–2195.

Ríos, J., Mas, F., & Menéndez, J. (2012). Aircraft final assembly line balancing and workload smoothing: a methodological analysis. In *Key engineering materials, Vol. 502* (pp. 19–24). Trans Tech Publ.

Şahin, M., & Kellegöz, T. (2019). A new mixed-integer linear programming formulation and particle swarm optimization based hybrid heuristic for the problem of resource investment and balancing of the assembly line with multi-manned workstations. *Computers & Industrial Engineering*, *133*, 107–120.

Sánchez, M., Cruz-Duarte, J. M., Ortiz-Bayliss, J. C., & Amaya, I. (2021). Sequence-based selection hyper-heuristic model via MAP-elites. *IEEE Access*, *9*, 116500–116527.

Scholl, A., & Becker, C. (2006). State-of-the-art exact and heuristic solution procedures for simple assembly line balancing. *European Journal of Operational Research*, *168*(3), 666–693.

Shadrokh, S., & Kianfar, F. (2007). A genetic algorithm for resource investment project scheduling problem, tardiness permitted with penalty. *European Journal of Operational Research*, *181*(1), 86–101.

Urgo, M. (2019). A branch-and-bound approach to schedule a no-wait flow shop to minimize the CVaR of the residual work content. *Computers & Industrial Engineering*, *129*, 67–75.

Xin, B., Li, Y., Yu, J., & Zhang, J. (2015). An adaptive BPSO algorithm for multi-skilled workers assignment problem in aircraft assembly lines. *Assembly Automation*, *35*(4), 317–328.

Zhang, Z., Tang, Q., & Qian, X. (2021). Integration of balancing and preventive maintenance in straight and U-shaped resource-dependent assembly lines: MILP model and memetic algorithm. *Applied Soft Computing*, *113*, Article 107773.

Zhu, H., Lu, Z., Lu, C., & Ren, Y. (2021). A reactive scheduling method for disturbances in aircraft moving assembly line. *International Journal of Production Research*, *59*(15), 4756–4772.